

Table of Contents

Báo cáo nghiên cứu: Các kỹ thuật xây dựng Notification trên Web

Phạm vi: đây là **báo cáo lý thuyết** (research report), chưa bao gồm code/POC. Mục tiêu là giúp team hiểu bản chất, ưu/nhược điểm, xu hướng sử dụng và cơ sở lựa chọn cho từng kỹ thuật. Các số liệu về xu hướng/thị phần/dự án thực tế đều có dẫn nguồn ở mục “Tham khảo” cuối mỗi phần; những chỗ không tìm được số liệu định lượng đáng tin cậy được ghi rõ là **qualitative/nhận định của cộng đồng kỹ thuật**, không phải số liệu khảo sát chính thức.

1. Notification là gì? Phân biệt với CRUD/Request-Response

Một hệ thống “notification” trên web **không phải là một công nghệ**, mà là sự kết hợp của 3 lớp độc lập:

Lớp	Vai trò	Ví dụ
Business event	Sự kiện nghiệp vụ xảy ra trong hệ thống	AppointmentCreated, TaskAssigned
Delivery mechanism (Communication)	Cách server đưa thông tin đến client	Polling, Long Polling, SSE, WebSocket, Web Push
Notification UI/Persistence	Cách hiển thị & lưu trữ cho người dùng	Toast, badge unread, DB record, OS notification

CRUD/request-response truyền thông là **client chủ động hỏi, server trả lời một lần rồi kết thúc**. Notification về bản chất là **server cân chủ động (hoặc gần như chủ động) đẩy thông tin về phía client mà client không hề yêu cầu tại đúng thời điểm đó** — đây là điểm khác biệt cốt lõi, vì giao thức HTTP nguyên bản được thiết kế theo mô hình pull (client hỏi trước), không phải push.

Do đó mọi kỹ thuật notification thực chất là các cách khác nhau để “giả lập” hoặc thực sự đạt được khả năng server → client push trên nền một giao thức vốn là pull-based (HTTP) hoặc dùng giao thức khác hỗ trợ push thật (WebSocket, Web Push).

2. Các kỹ thuật chính

2.1 Short Polling

Mô tả: Client gọi định kỳ (ví dụ mỗi 5–15 giây) một REST endpoint để hỏi “có gì mới không”. Server trả lời ngay lập tức, dù có dữ liệu mới hay không.

Đặc điểm nổi bật: - Không cần hạ tầng đặc biệt — chạy tốt trên mọi hosting, mọi proxy, mọi load balancer. - Độ trễ = tối đa bằng khoảng polling interval. - Số lượng request tăng tuyến tính theo số user $\times$ tần suất poll, phần lớn request trả về “không có gì mới” (empty response) — lãng phí băng thông và tải server.

Ưu điểm: - Triển khai đơn giản nhất, dùng HTTP thuần, không cần thư viện. - Hoạt động ổn định qua mọi firewall/proxy doanh nghiệp. - Dễ debug (chỉ là REST call bình thường, curl được). - Không cần giữ connection mở — không tốn socket/thread lâu dài trên server.

Nhược điểm: - Overhead cao nhất trong các kỹ thuật (nhiều request rỗng). - Độ trễ không thể thấp nếu muốn tiết kiệm tài nguyên (trade-off latency vs. cost). - Không phù hợp khi cần “gần real-time” ($<1s$) ở quy mô lớn.

Khi nên dùng: thông báo không khẩn cấp, tần suất thay đổi dữ liệu thấp, môi trường hạ tầng hạn chế (serverless function timeout ngắn, free hosting, mobile network kém ổn định).

Khi không nên dùng: chat, dashboard tài chính, hệ thống cần độ trễ dưới vài giây với lượng user lớn.

2.2 Long Polling

Mô tả: Client gửi request, nhưng server **giữ request mở** (không trả lời ngay) cho đến khi có dữ liệu mới hoặc hết timeout. Khi có phản hồi (dữ liệu hoặc timeout), client lập tức mở lại một request mới.

Đặc điểm nổi bật: - Giảm đáng kể số lượng request “rỗng” so với short polling, đồng thời độ trễ gần với real-time hơn (khi có event, response trả về gần như ngay lập tức). - Về bản chất vẫn là HTTP request/response — tương thích proxy tốt hơn WebSocket. - Server phải giữ connection/thread mở trong thời gian chờ — tốn tài nguyên hơn short polling nếu không dùng mô hình non-blocking I/O.

Ưu điểm: - Độ trễ thấp hơn short polling đáng kể. - Vẫn dùng HTTP thuần — tương thích hạ tầng cũ, proxy khó tính. - Là fallback tiêu chuẩn khi WebSocket/SSE bị chặn (đây là lý do Socket.IO vẫn giữ long-polling làm transport dự phòng).

Nhược điểm: - Lifecycle quản lý phức tạp hơn short polling: phải xử lý timeout server, timeout client, race condition khi nhiều request được mở đồng thời, dedupe event. - Nếu server dùng mô hình thread-per-request (không async), số lượng long-polling connection đồng thời có thể làm cạn kiệt thread pool. - Không có cơ chế chuẩn hóa qua trình duyệt (không có API native như EventSource/WebSocket) — phải tự viết retry logic.

Khi nên dùng: cần độ trễ thấp nhưng hạ tầng/mạng không đảm bảo hỗ trợ WebSocket/SSE (proxy doanh nghiệp khó tính); dùng làm fallback transport.

Khi không nên dùng: ứng dụng cần giao tiếp hai chiều liên tục, hoặc cần hàng nghìn kết nối đồng thời trên server không hỗ trợ non-blocking I/O tốt.

2.3 Server-Sent Events (SSE)

Mô tả: Client mở một kết nối HTTP duy nhất tới server với header `Accept: text/event-stream`; server giữ kết nối này mở và **liên tục stream** các sự kiện dạng text (`data: ... \n\n`) khi có dữ liệu mới. Chuẩn hóa qua HTML Living Standard, dùng qua EventSource API có sẵn trên trình duyệt.

Đặc điểm nổi bật: - Một chiều: server → client. Client **không thể gửi dữ liệu qua cùng kết nối này**. - EventSource có **auto-reconnect built-in**, hỗ trợ Last-Event-ID để phục hồi các event bị lỗi. - Chạy trên HTTP/HTTPS thuận → dễ đi qua CDN, proxy, load balancer hơn WebSocket (không cần handshake upgrade đặc biệt), nhưng vẫn có rủi ro bị **buffer bởi proxy trung gian** trong một số mạng doanh nghiệp — cần header `X-Accel-Buffering: no` (Nginx) hoặc tương đương. - Giới hạn lịch sử: qua HTTP/1.1, trình duyệt giới hạn 6 kết nối đồng thời/domain (nhiều tab cùng mở SSE tới cùng domain có thể bị nghẽn); qua HTTP/2 giới hạn này được giải quyết vì multiplexing.

Ưu điểm: - Rất đơn giản để triển khai (không cần thư viện, ~30 dòng code cho use case cơ bản). - Cơ chế reconnect + event-id built-in trong trình duyệt, giảm code phải tự viết. - Hoạt động tốt qua HTTP/S tiêu chuẩn — không cần WebSocket-aware load balancer. - Là lựa chọn mặc định hiện nay cho streaming AI completion (OpenAI, Anthropic, Google đều dùng SSE cho response dạng token-stream) vì bản chất một chiều khớp với nhu cầu.

Nhược điểm: - Chỉ hỗ trợ text UTF-8, không gửi được binary trực tiếp (phải encode base64 nếu cần). - Không có kênh client→server trên cùng kết nối (muốn gửi dữ liệu ngược lại phải mở request HTTP riêng). - iOS Safari có xu hướng ngắt kết nối SSE khi tab chuyển sang background — cần xử lý Page Visibility API. - Một số hạ tầng serverless/edge có giới hạn thời gian streaming (Vercel Hobby ~10s, Netlify Functions ~30s) khiến kết nối dài hạn bị cắt giữa chừng.

Khi nên dùng: notification server→client một chiều khi browser đang mở (dashboard, live feed, AI streaming, activity feed) — đúng như GitHub/Gitea đang dùng SSE cho notification endpoint `/user/events`.

Khi không nên dùng: cần client gửi dữ liệu trở lại real-time trên cùng kết nối (chat 2 chiều, game), cần truyền binary lớn.

2.4 WebSocket

Mô tả: Giao thức riêng (RFC 6455), bắt đầu bằng một HTTP handshake (`Upgrade: websocket`) rồi chuyển sang kết nối TCP **song công (full-duplex)** — cả client và server có thể gửi message bất kỳ lúc nào trên cùng một kết nối.

Đặc điểm nổi bật: - Hai chiều thực sự, độ trễ thấp nhất trong nhóm HTTP-based (~3ms nhanh hơn SSE theo benchmark, thường không đáng kể so với network RTT thực tế). - Phức tạp hơn đáng kể để vận hành production: cần tự quản lý reconnect, heartbeat (ping/pong) để phát hiện stale connection, xử lý multiple instance (một client kết nối vào server A, nhưng event lại phát sinh ở server B → cần cơ chế broadcast liên instance, thường dùng Redis

Pub/Sub hoặc message broker). - Cần load balancer/proxy “WebSocket-aware” (hỗ trợ HTTP upgrade + sticky session hoặc pub/sub layer), không phải mọi hạ tầng free-tier đều hỗ trợ tốt.

Ưu điểm: - Giao tiếp hai chiều thật, phù hợp cho tương tác liên tục tần suất cao (chat, collaborative editing, multiplayer game, typing indicator). - Một kết nối phục vụ được cả gửi lẫn nhận, giảm số lượng HTTP request rời rạc. - Hệ sinh thái thư viện trưởng thành (Socket.IO, ws, uWebSockets.js) hỗ trợ fallback, rooms, namespace.

Nhược điểm: - Chi phí vận hành/độ phức tạp cao nhất: heartbeat, reconnect logic, connection state, xử lý multi-instance. - Không debug được bằng curl thông thường; cần công cụ chuyên dụng. - Với hosting miễn phí có “spin down” khi idle, kết nối WebSocket bị ngắt bất kỳ lúc nào — phải xử lý reconnect thường xuyên hơn dự tính. - Dễ bị dùng “quá tay” — nhiều dự án chọn WebSocket cho use case chỉ cần một chiều (server-client), khiến hệ thống phức tạp không cần thiết so với SSE.

Khi nên dùng: giao tiếp hai chiều liên tục tần suất cao — chat, collaborative editing, multiplayer, presence/typing indicator, giao thức điều khiển AI agent (cần client gửi lệnh cancel/approve trong lúc server đang stream).

Khi không nên dùng: khi nhu cầu thực tế chỉ là server đẩy thông báo một chiều — dùng SSE sẽ đơn giản hơn nhiều với 90% lợi ích và 30% độ phức tạp (nhận định phổ biến trong cộng đồng, xem TechPlained/CodeScoop bên dưới).

2.5 Web Push (Push API + Service Worker + Notification API)

Đây là nhóm kỹ thuật khác hẳn về bản chất so với 4 kỹ thuật trên: mục tiêu không phải “cập nhật UI khi tab đang mở” mà là **đưa thông báo tới người dùng ngay cả khi họ không mở trang web**, tương tự push notification trên mobile app.

Các thành phần bắt buộc: 1. **Service Worker** — một script chạy nền, độc lập với vòng đời của tab trình duyệt, bắt buộc phải chạy trên **secure context (HTTPS)**. 2. **Push API** — cho phép Service Worker đăng ký một PushSubscription (endpoint duy nhất do Push Service của trình duyệt — ví dụ FCM cho Chrome, Mozilla Push cho Firefox — cấp phát). 3. **VAPID (Voluntary Application Server Identification)** — cặp khóa public/private để application server tự xác thực với Push Service mà không cần đăng ký tài khoản với từng hãng trình duyệt. 4. **Notification API** — cho phép Service Worker hiển thị notification cấp hệ điều hành khi nhận được push event.

Luồng hoạt động:

Browser → xin permission → Service Worker đăng ký PushSubscription (qua Push Service của browser)

→ gửi subscription về Application Server để lưu

Application Server → khi có event → gửi payload (mã hoá) tới Push Service kèm VAPID token

Push Service (Google/Mozilla/Apple...) → đánh thức Service Worker của trình duyệt (kể cả khi tab đóng)

Service Worker → nhận 'push' event → gọi Notification API để hiển thị OS-level notification

Điểm khác biệt quan trọng với SSE/WebSocket: Web Push **không đi trực tiếp từ server của bạn tới browser** — nó luôn đi qua Push Service trung gian do trình duyệt vận hành (FCM, Mozilla Autopush...), nên độ trễ và tỷ lệ delivery còn phụ thuộc vào dịch vụ bên thứ ba này, không nằm hoàn toàn trong tầm kiểm soát của hệ thống.

Ưu điểm: - Hoạt động cả khi tab/trang web đóng (miễn Service Worker vẫn được đăng ký và permission còn hiệu lực) — đây là khả năng duy nhất trong nhóm 5 kỹ thuật đạt được “offline/background delivery”. - Hiển thị notification cấp hệ điều hành — tăng khả năng người dùng chú ý. - Không cần giữ kết nối mở liên tục từ phía client — tiết kiệm tài nguyên hơn WebSocket/SSE cho notification không thường xuyên.

Nhược điểm: - Yêu cầu HTTPS bắt buộc, yêu cầu permission người dùng chủ động chấp thuận (nhiều người dùng từ chối). - Subscription có thể hết hạn/không hợp lệ (expired/invalid subscription) — server phải xử lý lỗi 404/410 khi gửi và tự dọn subscription cũ. - Payload có giới hạn kích thước (thường ~4KB tùy Push Service) và phải mã hoá theo chuẩn aes128gcm. - Trải nghiệm/hành vi không đồng nhất tuyệt đối giữa các trình duyệt/OS (đặc biệt Safari/iOS có những giới hạn riêng, ví dụ Web Push trên Safari chỉ được hỗ trợ đầy đủ từ iOS 16.4 trở đi khi web app được “Add to Home Screen”). - Không phù hợp cho update tần suất cao hoặc yêu cầu hiển thị ngay lập tức trong UI (Web Push tối ưu cho thông báo rời rạc, không phải luồng dữ liệu liên tục).

Khi nên dùng: thông báo cần đến tay người dùng dù họ không mở tab (đơn hàng đã giao, tin nhắn mới khi offline, nhắc lịch), tăng khả năng “gọi người dùng quay lại” ứng dụng.

Khi không nên dùng: cập nhật UI liên tục khi người dùng đang thao tác trong app (nên dùng SSE/WebSocket); trường hợp cần đảm bảo 100% delivery tuyệt đối (Web Push không đảm bảo delivery, chỉ “best-effort” qua push service bên thứ ba).

2.6 Các kỹ thuật liên quan đáng cân nhắc (bổ sung)

Kỹ thuật	Lý do đưa vào	Ghi chú
MQTT (qua WebSocket)	Giao thức pub/sub nhẹ, phổ biến cho IoT/telemetry, đôi khi được dùng làm transport cho mobile push nội bộ (Facebook Messenger từng dùng MQTT)	Không phải chuẩn web-native, cần broker riêng (Mosquitto, EMQX...)
WebTransport (HTTP/3)	Kế thừa tiềm năng của WebSocket dựa trên QUIC, hỗ trợ multiplexing, giảm head-of-line blocking	Tính đến đầu 2026 vẫn ở dạng W3C Working Draft, hỗ trợ trình duyệt còn hạn chế (chủ yếu Chromium) — chưa nên dùng làm giải pháp chính cho production mà không có fallback

Kỹ thuật	Lý do đưa vào	Ghi chú
Managed real-time services (Ably, Pusher, PubNub)	Không phải kỹ thuật transport mới mà là dịch vụ trừu tượng hoá WebSocket/SSE, cung cấp guaranteed delivery, presence, history, horizontal scaling sẵn	Đáng cân nhắc khi đội ngũ nhỏ, không muốn tự vận hành hạ tầng real-time

3. Bảng so sánh tổng hợp

Đây là **đánh giá định tính (qualitative assessment)** tổng hợp từ nhiều nguồn kỹ thuật (không phải benchmark đo đạc trực tiếp của nhóm), trừ các số liệu có trích dẫn cụ thể.

Tiêu chí	Short Polling	Long Polling	SSE	WebSocket	Web Push
Direction	Client→Server (pull)	Client→Server (pull, giữ mở)	Server→Client	Bidirectional	Server→Client (qua push service)
Real-time capability	Thấp	Trung bình–Cao	Cao	Cao nhất	Không đồng bộ (best-effort)
Latency	= polling interval	Gần tức thời khi có event	Gần tức thời	Thấp nhất (~ms)	Biến thiên, phụ thuộc push service
Connection overhead	Thấp/request nhưng nhiều request	Trung bình (giữ mở)	Trung bình (1 kết nối dài)	Trung bình (1 kết nối dài, 2 chiều)	Không giữ kết nối liên tục
Network efficiency	Kém (nhiều response rộng)	Khá	Tốt	Tốt	Rất tốt (chỉ gửi khi có event)
Server resource usage	Thấp/request, cao khi scale user	Cao nếu blocking I/O	Trung bình (nhiều socket mở)	Trung bình–Cao (2 chiều, cần heartbeat)	Thấp (server không giữ kết nối)
Client complexity	Thấp	Trung bình	Thấp (EventSource built-in)	Trung bình–Cao	Cao (Service Worker, permission, VAPID)
Backend complexity	Thấp	Trung bình	Thấp–Trung bình	Cao (đặc biệt multi-instance)	Trung bình (mã hoá payload, quản lý)

Tiêu chí	Short Polling	Long Polling	SSE	WebSocket	Web Push
Reconnection	Tự implement	Tự implement	Built-in (EventSource)	Tự implement (không built-in)	subscription) Không áp dụng (không giữ kết nối)
Offline/ background support	Không	Không	Không (mất khi đóng tab)	Không	Có
Browser-closed support	Không	Không	Không	Không	Có
Bidirectional	Không	Không (nửa vôi)	Không	Có	Không
Scalability (nhiều instance)	Dễ (stateless)	Trung bình	Cần sticky/pub-sub khi nhiều instance	Cần Redis/broker khi nhiều instance	Dễ (không giữ state kết nối)
Reliability/ Guaranteed delivery	Có thể tự đảm bảo qua DB query	Có thể tự đảm bảo	Cần tự thêm event-id + replay	Cần tự thêm ack/replay	Best-effort, không đảm bảo
Persistence support	Không liên quan trực tiếp	Không liên quan trực tiếp	Không liên quan trực tiếp	Không liên quan trực tiếp	Không liên quan trực tiếp (persistence là tầng riêng, DB)
Security complexity	Thấp (auth như REST)	Thấp	Trung bình (auth cho long-lived connection)	Trung bình–Cao (auth khi handshake + theo message)	Trung bình (VAPID, mã hoá payload)
Infrastructure complexity	Thấp	Thấp–Trung bình	Trung bình	Cao	Trung bình (cần Push Service bên thứ ba)
Hosting compatibility (free tier)	Rất tốt	Tốt (nếu server hỗ trợ async)	Tốt (chú ý timeout serverless)	Hạn chế (idle spin-down, cần WS-aware)	Tốt (không cần giữ kết nối lâu)
Implementation	Thấp	Trung bình	Thấp–Trung bình	Cao	Cao (nhiều thành phần)

Tiêu chí	Short Polling	Long Polling	SSE	WebSocket	Web Push
complexity					
Best use cases	Thông báo không khẩn, traffic nhỏ	Fallback khi WS/SSE bị chặn	Dashboard, feed, AI streaming, notification server—client	Chat, collaborative editing, game, agent control	Thông báo khi user offline/tab đóng
Nhược điểm chính	Overhead, độ trễ cao	Lifecycle phức tạp, tốn thread nếu blocking	Không gửi được client→server, 1 chiều	Vận hành phức tạp, cần broker khi scale	Không đảm bảo delivery, cần permission, phức tạp setup

4. Xu hướng sử dụng thực tế & dự án tiêu biểu (có dẫn nguồn)

4.1 WebSocket

- **Slack:** mỗi client Slack duy trì một kết nối WebSocket liên tục tới Gateway Server (qua Envoy làm edge proxy); tại giờ cao điểm Slack duy trì **hơn 5 triệu WebSocket session đồng thời**, hệ thống gồm Channel Server / Gateway Server / Admin Server / Presence Server, và message được phát tán tới các client qua broadcast trên WebSocket. (Nguồn: *Slack Engineering — “Real-time Messaging”*, slack.engineering; *ByteByteGo phân tích lại kiến trúc này*, 2025)
- **Discord:** mỗi guild (server) là một tiến trình Elixir riêng làm điểm định tuyến trung tâm cho kết nối realtime (thường qua WebSocket gateway). (Nguồn: *tổng hợp kiến trúc Discord*, [snowan study-notes](https://snowan.study-notes), 2026)
- Theo cộng đồng, WebSocket vẫn là lựa chọn hàng đầu khi cần **tương tác 2 chiều tần suất cao**: chat, collaborative editing, multiplayer game — đây là pattern lặp lại nhất quán trong các bài phân tích kỹ thuật 2025–2026 (APIScout, WebSocket.org, TechPlained).

4.2 SSE

- **GitHub / Gitea:** dùng endpoint SSE (/user/events, chạy qua SharedWorker để chia sẻ 1 kết nối cho nhiều tab) để đẩy sự kiện repo activity, notification tới client — đây là dẫn chứng thực tế trực tiếp trong mã nguồn/issue tracker của Gitea (dự án mã nguồn mở tương thích GitHub API). (Nguồn: *GitHub Issue go-gitea/gitea #36937*, 2026)
- **LLM streaming API:** OpenAI, Anthropic, Google đều dùng SSE để stream token phản hồi — vì bản chất luồng dữ liệu một chiều (model – client) khớp hoàn toàn với mô hình SSE; SSE tận dụng được Authorization: Bearer header chuẩn REST, không cần giao thức riêng. (Nguồn: *AllDevToolsHub*, “*SSE vs WebSockets vs Long Polling 2026*”)
- Xu hướng gần đây (2026): một số framework AI (Vercel AI SDK) đang **chuyển từ SSE sang giao thức pluggable/WebSocket** cho các luồng agent cần tương tác 2 chiều (cancel, approve tool call) — cho thấy SSE vẫn là mặc định tốt cho streaming một chiều

nhưng không phải giải pháp vạn năng khi nhu cầu chuyển sang bidirectional. (Nguồn: websocket.org, bài viết của CEO Ably, 2026)

4.3 Long Polling

- Lịch sử: nhiều hệ thống chat/email cũ (bao gồm Gmail chat, XMPP BOSH) từng dùng long polling trước khi WebSocket được chuẩn hóa rộng rãi (~2011).
- Hiện tại: **Socket.IO** vẫn giữ long-polling làm transport fallback mặc định khi WebSocket handshake thất bại — đây là bằng chứng nó vẫn có vai trò thực tế, không phải công nghệ “chết”. (Nguồn: RxDB, “WebSockets vs SSE vs Long-Polling”, 2026; npm-compare.com về Socket.IO)
- Use case còn tồn tại rõ nhất: **môi trường doanh nghiệp (B2B) sau proxy/firewall khó tính** chặn WebSocket upgrade và SSE stream như traffic bất thường — long polling vẫn là lựa chọn “chạy được ở mọi nơi”. (Nguồn: APIScout, “Real-Time APIs: WebSockets vs SSE vs Long Polling 2026”)

4.4 Web Push

- Tỷ lệ opt-in cho web push (không phải mobile app push) dao động phổ biến **10–15%** theo tổng hợp Gravitec/Sleeknote; một số nguồn khác báo cáo con số thấp hơn (~5–6%) tùy ngành (PushPushGo, 2026) — chênh lệch số liệu giữa các nhà cung cấp dịch vụ cho thấy đây là **thị trường phân mảnh, số liệu phụ thuộc nguồn đo**, cần thận trọng khi trích dẫn tuyệt đối.
- **~95% subscriber nhận push qua trình duyệt Chrome** theo nhiều nguồn thống kê độc lập (WiserNotify, Gravitec) — phản ánh thực tế Chrome thống trị thị phần trình duyệt hỗ trợ Web Push tốt nhất.
- **Safari trên iOS chỉ hỗ trợ Web Push đầy đủ từ iOS 16.4 (2023)**, và yêu cầu web app được thêm vào Home Screen trong nhiều trường hợp — đây là giới hạn quan trọng cần biết khi thiết kế hệ thống nhắm tới người dùng iOS. (Nguồn: sci-tech-today.com, tổng hợp thống kê push notification 2026)
- Thị trường phần mềm push notification (bao gồm cả mobile) được ước tính tăng trưởng mạnh (nhiều nguồn báo cáo tăng trưởng hai chữ số theo năm), cho thấy xu hướng đầu tư vào kênh này tiếp tục mở rộng, dù các số liệu thị trường cụ thể (tỷ USD) giữa các công ty nghiên cứu thị trường có sự khác biệt lớn — nên xem là ước lượng ngành, không phải số liệu chính xác tuyệt đối.

4.5 Mức độ phổ biến thư viện (proxy cho xu hướng adoption)

Theo dữ liệu tải xuống npm (npm-trends.com, cập nhật gần nhất trong nghiên cứu này): - **ws** (thư viện WebSocket cấp thấp cho Node.js): khoảng **85–131 triệu lượt tải/tuần** (dao động theo thời điểm đo) — con số cao một phần vì ws là dependency ẩn của rất nhiều framework khác (Next.js, Vite, các dev server...), không chỉ vì được dùng trực tiếp cho notification. - **socket.io**: khoảng **5–13 triệu lượt tải/tuần**, ~60.000+ GitHub stars — vẫn là thư viện realtime bidirectional phổ biến nhất khi cần API cấp cao (rooms, reconnection, fallback tự động). - Xu hướng 2026 (theo PkgPulse, tổng hợp hệ sinh thái npm): kiến trúc thực tế phổ biến nhất là **kết hợp cả 3**: SSE cho luồng một chiều không cần thư viện, WebSocket/Socket.IO tự host cho tương tác 2 chiều quy mô vừa, và dịch vụ managed (Ably/Pusher/PartyKit) khi cần scale lớn mà không muốn tự vận hành hạ tầng realtime.

Nhận định tổng quát rút ra từ các nguồn trên (được nhiều bài viết kỹ thuật độc lập đồng thuận, không phải số liệu khảo sát chính thức một tổ chức duy nhất): phần lớn “yêu cầu real-time” trong thực tế là **một chiều (server–client)**, và SSE ngày càng được khuyến nghị làm mặc định cho notification thay vì WebSocket, WebSocket chỉ nên là lựa chọn khi nhu cầu bidirectional thực sự tồn tại. Đây là quan điểm lặp lại nhất quán ở nhiều nguồn độc lập (WebSocket.org do CEO Ably viết, TechPlained, CodeScoop/DEV Community, APIScout) — không phải quan điểm của riêng một bên.

5. Requirement Matrix — cơ sở để chọn kỹ thuật

Trước khi chọn công nghệ, cần trả lời các câu hỏi sau cho hệ thống cụ thể:

Câu hỏi	Ảnh hưởng tới lựa chọn
Có cần real-time không? Latency chấp nhận được bao nhiêu?	<1s – SSE/WebSocket; vài giây–vài phút – Polling đủ
Server–Client hay Client–Server?	1 chiều – SSE; 2 chiều tần suất cao – WebSocket
Browser có thể đóng/tab background không, có cần nhận khi đó?	Có – bắt buộc cân nhắc Web Push
Notification có được phép mất không?	Không – cần persistence + recovery (event-id/Last-Event-ID, DB query after=lastEventId), không dựa hoàn toàn vào persistent connection
Bao nhiêu concurrent user, một hay nhiều server instance?	Nhiều instance + WebSocket/SSE – cần Redis Pub/Sub hoặc message broker để broadcast liên instance
Hosting có free-tier/serverless với giới hạn timeout không?	Có – ưu tiên Polling/SSE ngắn hạn hoặc Web Push; WebSocket/SSE dài hạn cần hosting hỗ trợ persistent connection thực sự
Có cần OS-level notification khi không mở web?	Có – chỉ Web Push đáp ứng được

6. Bảng khuyến nghị theo yêu cầu (giả thuyết ban đầu — không phải kết luận tuyệt đối)

Requirement	Đề xuất	Lý do
Thông báo không khẩn cấp, traffic nhỏ	Polling	Đơn giản nhất, chi phí hạ tầng thấp nhất
Server–Client một chiều, gần real-time, browser đang mở	SSE	Đơn giản, auto-reconnect built-in, đủ nhanh cho phần lớn use case notification
Giao tiếp 2 chiều tần suất cao	WebSocket	Chỉ WebSocket cho phép

Requirement	Đề xuất	Lý do
(chat, collab, game)		bidirectional thật trên 1 kết nối
Cần notification khi browser đóng/không mở tab, OS-level	Web Push	Duy nhất hỗ trợ background/offline delivery
Môi trường mạng doanh nghiệp hạn chế, cần chạy mọi nơi	Long Polling	Là HTTP thuần, ít bị proxy/firewall chặn nhất

Lưu ý quan trọng: Đây là các “starting hypothesis” dựa trên nghiên cứu lý thuyết + tài liệu ngành. Trong thực tế, phần lớn hệ thống production **kết hợp nhiều kỹ thuật cùng lúc** (ví dụ: SSE cho in-app real-time + Web Push cho khi user offline + REST cho lịch sử/đọc/chưa đọc), và lựa chọn cuối cùng cần được xác nhận bằng implementation + benchmark thực tế trên chính hạ tầng/traffic pattern của hệ thống, không nên chọn chỉ vì một công nghệ đang “phổ biến” hoặc “nghe có vẻ real-time hơn”.

7. Nguồn tham khảo chính

- Slack Engineering — “Real-time Messaging”, slack.engineering/real-time-messaging/
- ByteByteGo — “How Slack Supports Billions of Daily Messages” (2025)
- GitHub go-gitea/gitea, Issue #36937 — SSE notification proposal (2026)
- WebSocket.org (Abyl) — “WebSocket vs SSE: Which One Should You Use?” và “WebSocket vs HTTP, SSE, MQTT, WebRTC & More” (2026)
- Abyl — “A WebSocket alternatives: Exploring realtime protocols”
- RxDB — “WebSockets vs Server-Sent-Events vs Long-Polling vs WebRTC vs WebTransport”
- APIScout — “Real-Time APIs: WebSockets vs SSE vs Long Polling 2026”
- TechPlained, CodeScoop (DEV Community) — so sánh WebSocket/SSE/Long Polling thực chiến
- npmtrends.com — số liệu tải xuống ws, socket.io
- PkgPulse — “Best npm Packages for Realtime in 2026”, “Socket.IO vs ws vs uWebSockets.js 2026”
- Sleeknote, Gravitec, PushPushGo, sci-tech-today.com, WiserNotify — thông kê Web Push (opt-in rate, trình duyệt, iOS support)
- MDN Web Docs — tham chiếu chuẩn cho EventSource, WebSocket, Push API, Notification API (dùng để kiểm chứng hành vi kỹ thuật nền tảng, không trích dẫn số liệu)

Ghi chú phương pháp luận: các số liệu thị trường/thông kê push notification giữa các nguồn marketing (Sleeknote, WiserNotify, PushPushGo...) có sự chênh lệch đáng kể do khác phương pháp đo — báo cáo này trình bày khoảng giá trị và ghi rõ nguồn thay vì chọn một con số duy nhất làm “sự thật tuyệt đối”. Các claim về kiến trúc production (Slack, GitHub/Gitea) có độ tin cậy cao hơn vì đến từ chính đội ngũ kỹ thuật/mã nguồn của dự án.